第 5 天 · 11/6 · 今日知识点回顾

函数：参数、返回值、作用域、lambda

目标：把一段 30 行的流水账脚本拆成 4 个函数，每个函数只干一件事，且能说清参数是「值传递」还是「引用传递」。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 把一段 30 行的流水账脚本拆成 4 个函数，每个函数只干一件事，且能说清参数是「值传递」还是「引用传递」。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：6 集 • 合计约 58 分钟（边看边在本机敲一遍）

P51	函数的初体验	约 12 分钟
P52	函数的基础定义语法	约 9 分钟
P54	函数的传入参数	约 10 分钟
P56	函数的返回值定义语法	约 8 分钟
P60	变量在函数中的作用域	约 10 分钟
P84	lambda匿名函数	约 9 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://raw.githubusercontent.com/jackfrued/Python-100-Days/master/Day01-20/14.函数和模块.md

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/6 • def 定义，return 返回

- 一句话：def 名字(参数): 下面缩进写函数体；return 把结果交出去，没写 return 就返回 None。
- 代码：

```
def bmi(height_m, weight_kg):
return weight_kg / height_m ** 2
print(bmi(1.75, 70))        # 22.857142857142858
print(round(bmi(1.75, 70), 2))   # 22.86
print(bmi(weight_kg=70, height_m=1.75))   # 22.857... ← 关键字参数，顺序无所谓
```

★ 忘了 return 是最隐蔽的 bug：函数体里 print 了结果，但函数本身返回 None，外面 a = f() 之后 a 就是 None，后面 a + 1 报 TypeError。

知识点 2/6 • 默认参数与可变参数

- 一句话：默认参数写 def f(a, b=10)；可变参数写 *args（元组）和 **kwargs（字典）。
- 代码：

```
def to_num(s, default=0.0):
try:
return float(str(s).replace('¥', '').replace(',', ''))
except (TypeError, ValueError):
return default
print(to_num('¥1,899.00'))   # 1899.0
print(to_num(''))            # 0.0
print(to_num(None, -1))      # -1
```

知识点 2/6 • 默认参数与可变参数（续）

★ 默认参数只在「定义时」求值一次。def f(x, bucket=[]) 里的 bucket 会被所有调用

- 共享，追加几次就串了 —— 要用 None 兜：def f(x, bucket=None): bucket = bucket or []

知识点 3/6 • 作用域：函数里改不了外面的数字，但能改列表

- 一句话：函数内赋值会新建局部变量；global 才能改全局，但尽量别用。
- 代码：

```
counter = 0
def bump():
global counter
```

- counter += 1bump(); bump()

知识点 3/6 • 作用域：函数里改不了外面的数字，但能改列表（续）

```
print(counter)          # 2
nums = [1, 2, 3]
def add(x):
x.append(4)         # 没有赋值，直接改对象 → 外面也变
add(nums)
print(nums)             # [1, 2, 3, 4]
```

★ 「值传递 / 引用传递」这个面试题的正确答法：Python 传的是「对象引用」。

- 函数内重新赋值不影响外面，原地修改（append / sort / [0]=）会影响外面。

知识点 4/6 • lambda：一行的匿名函数，专门喂给 key

- 一句话：lambda 参数: 表达式，通常只用在 sorted / max / min / map / filter 的 key 上。
- 代码：

```
rows = [{'city': '北京', 'amount': 1899.0},
```

- {'city': '上海', 'amount': 699.0}]rows.sort(key=lambda r: r['amount'], reverse=True)

知识点 4/6 • lambda：一行的匿名函数，专门喂给 key（续）

```
print([r['city'] for r in rows])          # ['北京', '上海']
print(max(rows, key=lambda r: r['amount'])['city'])   # 北京
```

> **★ lambda 里不能写语句（不能 print、不能赋值、不能 if-else 的多分支），只能是一个表达式。超过一行的逻辑就老老实实写 def。**

知识点 5/6 • map / filter / 列表推导式，选一个就行

- 一句话：三者都能做转换和筛选，团队代码里推导式最易读，map/filter 次之。
- 代码：

```
nums = [1, 2, 3, 4, 5]
print([n * 2 for n in nums])                # 推导式（推荐）
print(list(map(lambda n: n * 2, nums)))     # map
print([n for n in nums if n % 2 == 0])      # 筛选
print(list(filter(lambda n: n % 2 == 0, nums)))
```

★ map/filter 返回的是迭代器，不是列表 —— 直接 print 会得到 <map object at 0x...>，要么套 list()，要么改成推导式。这个坑在 Python 3 里天天有人踩。

知识点 6/6 • 写函数的三条硬规矩

- 一句话：一个函数只干一件事；参数不超过 4 个；有副作用（读写文件/打印）要写进名字。
- 代码：

```
def load_orders(path): ...          # 读文件
def clean_amount(rows): ...         # 清洗
def summarize_by_city(rows): ...    # 汇总
def save_csv(rows, path): ...       # 写出
```

- （这四步就是第 14 天 W5 验收脚本的骨架。今天先按这个结构把昨天的练习拆成函数，明天开始每个函数单独能测。）

代码速查：今天这几段，要在本机跑通（1/5）

把第 3 天的练习脚本重写成 4 个函数，每个函数单独能跑：

```
import csv, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
def load_orders(path):
```

```
    with open(path, encoding='utf-8-sig', newline='') as f:
```

return list(csv.DictReader(f))

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

```
def to_num(s, default=0.0):
```

```
    try:
```

```
        return float(str(s).replace('¥', '').replace(',', ''))
```

```
    except (TypeError, ValueError):
```

```
        return default
```

```
def summarize_by_city(rows, status='paid'):
```

```
    agg = {}
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/5）

for r in rows:

```
        if r['status'].strip().lower() != status:
```

continue

city = r['city'] or '（未知）'

```
        agg[city] = agg.get(city, 0.0) + to_num(r['amount'])
```

```
    return dict(sorted(agg.items(), key=lambda kv: kv[1], reverse=True))
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

```
if __name__ == '__main__':
```

```
    rows = load_orders(DATA / 'orders_dirty.csv')
```

print(len(rows), '行')

```
    for city, total in list(summarize_by_city(rows).items())[:5]:
```

print(f'{city}\t{total:,.2f}')

验收（做到才算过关）：

① 四个函数各自能单独调用、单独验证；

代码速查：今天这几段，要在本机跑通（5/5）

② `if __name__ == '__main__':` 那一行的作用能讲清楚（模块被 import 时不执行）；

③ 把 summarize_by_city 改成按 category 汇总，只改一行参数。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

共享，追加几次就串了 —— 要用 None 兜：def f(x, bucket=None): bucket = bucket or []◆ 知识点 3 | 作用域：函数里改不了外面的数字，但能改列表一句话：函数内赋值会新建局部变量；global 才能改全局，但尽量别用。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：lambda 里不能写语句（不能 print、不能赋值、不能 if-else 的多分支），只能是一个表达式。超过一行的逻辑就老老实实写 def。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 6 天 • 面向对象 + 模块与包 + 常用标准库

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-06-第05天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data